

v.31 Interop Audit



May 15, 2026

Table of Contents

Table of Contents	2
Summary	4
Scope	5
System Overview	10
Interoperability Updates	10
Settlement and Migration Changes	11
Base Token Architecture Changes	11
Asset chainBalance Migration	11
Security Model and Trust Assumptions	12
Notes on the Test Suite	14
Considerations on Compromised Chains and Malicious Settlement Layers	14
Isolation vs Propagation of Failures	15
Cross-Chain Liveness Considerations	16
Medium Severity	18
M-01 Missing Pause Enforcement in L2AssetRouter.bridgehubDepositBaseToken	18
Low Severity	18
L-01 Incorrect Length Validation in ERC-7930 Address Parsing	18
L-02 Missing Early Validation of ERC-7930 Bundle Addresses	19
L-03 Missing Validation of interopCallValue in Indirect Calls	20
L-04 Potential Double Accounting in Base Token Bridging Flow	21
L-05 Missing Timestamp Consistency Check in Committer.verifyBatchTimestamp	21
L-06 Overly Broad Access Control Permissions Across Multiple Contracts	22
L-07 Missing Input Validation	23
L-08 Asymmetric Base Token Supply Cap Between ZKOS and Non-ZKOS Chains	24
Notes & Additional Information	25
N-01 Redundant Branching in InteropCenter._ensureCorrectTotalValue	25
N-02 Inconsistencies in Fee Price Adjustment Logic	26
N-03 InteropHandler Does Not Explicitly Inherit IERC7786Recipient	27
N-04 Missing Revert on Invalid BUNDLE_IDENTIFIER in Interop Message Handling	27
N-05 Ambiguous Accounting for Base Tokens in Asset Trackers	28
N-06 Inconsistent Restoration of pausedDepositsTimestamp Across Migration Flows	28
N-07 Missing Deposit Pause Validation on Receiving Settlement Layer	29
N-08 Hardcoded originChainId in Base Token Finalization May Be Misleading	30
N-09 Naming Inconsistencies	30

N-10 Redundant Code and Minor Structural Improvements	31
N-11 Typographical Errors	32
N-12 Redundant Logic and Minor Code Simplifications	32
N-13 Misleading or Incomplete Comments	33
N-14 Missing Deadline Initialization Check in GovernanceUpgradeTimer	34
N-15 Unused Code	34
Client Reported	35
CR-01 Interop Bundles Can Be Claimed On Chains Settling On L1	35
CR-02 ProtocolUpgradeHandler cannot freeze chains under the ZKOS ChainTypeManager	36
Conclusion	37
Appendix	38
Issue Classification	38

Summary

Type	L2, Interoperability	Total Issues	26 (24 resolved)
Timeline	From 2026-03-26 To 2026-05-05	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	1 (1 resolved)
		Low Severity Issues	8 (8 resolved)
		Notes & Additional Information	15 (13 resolved)
		Client Reported Issues	2 (2 resolved)

Scope

OpenZeppelin performed a diff audit of the "matter-labs/era-contracts" repository with base commit [babbed5](#) and head commit [4271d0a](#).

In scope were the following files:

```
contracts
├── da-contracts/contracts
│   ├── BlobsL1DAValidatorZKsync0S.sol
│   ├── CalldataDA.sol
│   ├── DAContractsErrors.sol
│   ├── EIP7702Checker.sol
│   ├── IEIP7702Checker.sol
│   └── RollupL1DAValidator.sol
├── l1-contracts/contracts
│   ├── bridge
│   │   ├── BridgeHelper.sol
│   │   ├── BridgedStandardERC20.sol
│   │   ├── L1BridgeContractErrors.sol
│   │   ├── L1ERC20Bridge.sol
│   │   ├── L1Nullifier.sol
│   │   ├── L2SharedBridgeLegacy.sol
│   │   ├── L2WrappedBaseToken.sol
│   │   ├── L2WrappedBaseTokenStore.sol
│   │   ├── UpgradeableBeaconDeployer.sol
│   │   ├── asset-router
│   │   │   ├── AssetRouterBase.sol
│   │   │   ├── IAssetRouterBase.sol
│   │   │   ├── IAssetRouterShared.sol
│   │   │   ├── IL1AssetRouter.sol
│   │   │   ├── IL2AssetRouter.sol
│   │   │   ├── L1AssetRouter.sol
│   │   │   └── L2AssetRouter.sol
│   │   ├── asset-tracker
│   │   │   ├── AssetTrackerBase.sol
│   │   │   ├── AssetTrackerErrors.sol
│   │   │   ├── GWAssetTracker.sol
│   │   │   ├── IAssetTrackerBase.sol
│   │   │   ├── IAssetTrackerDataEncoding.sol
│   │   │   ├── IGWAssetTracker.sol
│   │   │   ├── IL1AssetTracker.sol
│   │   │   ├── IL2AssetTracker.sol
│   │   │   ├── L1AssetTracker.sol
│   │   │   ├── L2AssetTracker.sol
│   │   │   └── LegacySharedBridgeAddresses.sol
│   │   └── interfaces
│   │       ├── IL1AssetHandler.sol
│   │       └── IL1CrossChainSender.sol
```

- IL1ERC20Bridge.sol
 - IL1Nullifier.sol
 - IL2CrossChainSender.sol
 - IL2WrappedBaseToken.sol
- ntv
 - IL1NativeTokenVault.sol
 - IL2NativeTokenVault.sol
 - INativeTokenVaultBase.sol
 - L1NativeTokenVault.sol
 - L2NativeTokenVault.sol
 - L2NativeTokenVaultZKOS.sol
 - NativeTokenVaultBase.sol
- common
 - Config.sol
 - L1ContractErrors.sol
 - MessageVerification.sol
 - Messaging.sol
 - interfaces
 - IAccountCodeStorage.sol
 - IL2ContractDeployer.sol
 - IMessageVerification.sol
 - ISystemContext.sol
 - l2-helpers
 - IBaseToken.sol
 - IL2ToL1Messenger.sol
 - IL2ToL1MessengerEra.sol
 - L2ContractAddresses.sol
 - L2ContractHelper.sol
 - L2ContractInterfaces.sol
 - SystemContractsCaller.sol
 - libraries
 - DataEncoding.sol
 - DynamicIncrementalMerkle.sol
 - DynamicIncrementalMerkleMemory.sol
 - FullMerkle.sol
 - FullMerkleMemory.sol
 - Merkle.sol
 - MessageHashing.sol
 - TransientPrimitives.sol
 - ZKSyncOSBytecodeInfo.sol
- core
 - bridgehub
 - BridgehubBase.sol
 - IBridgehubBase.sol
 - IL1Bridgehub.sol
 - IL2Bridgehub.sol
 - L1Bridgehub.sol
 - L1BridgehubErrors.sol
 - L2Bridgehub.sol
 - chain-asset-handler
 - ChainAssetHandlerBase.sol
 - IChainAssetHandler.sol
 - IChainAssetHandlerShared.sol
 - IL1ChainAssetHandler.sol
 - IL2ChainAssetHandler.sol
 - L1ChainAssetHandler.sol

- └─ L2ChainAssetHandler.sol
- └─ chain-registration
 - └─ ChainRegistrationSender.sol
 - └─ IChainRegistrationSender.sol
- └─ ctm-deployment
 - └─ CTMDeploymentTracker.sol
 - └─ ICTMDeploymentTracker.sol
- └─ message-root
 - └─ IL1MessageRoot.sol
 - └─ IMessageRoot.sol
 - └─ L1MessageRoot.sol
 - └─ L2MessageRoot.sol
 - └─ MessageRootBase.sol
- └─ governance
 - └─ ChainAdmin.sol
 - └─ Governance.sol
 - └─ IServerNotifier.sol
 - └─ PermanentRestriction.sol
- └─ interop
 - └─ AttributesDecoder.sol
 - └─ IERC7786Attributes.sol
 - └─ IERC7786GatewaySource.sol
 - └─ IERC7786Recipient.sol
 - └─ IInteropCenter.sol
 - └─ IInteropHandler.sol
 - └─ IL2InteropRootStorage.sol
 - └─ InteropCenter.sol
 - └─ InteropDataEncoding.sol
 - └─ InteropErrors.sol
 - └─ InteropHandler.sol
 - └─ L2InteropRootStorage.sol
 - └─ L2MessageVerification.sol
- └─ l2-system
 - └─ BaseTokenHolder.sol
 - └─ L2BaseTokenBase.sol
 - └─ era
 - └─ L2BaseTokenEra.sol
 - └─ interfaces/IL2BaseTokenEra.sol
 - └─ interfaces
 - └─ IBaseTokenHolder.sol
 - └─ IL2BaseTokenBase.sol
 - └─ zksync-os
 - └─ L1MessageGasLib.sol
 - └─ L1MessengerZKOS.sol
 - └─ L2BaseTokenZKOS.sol
 - └─ SystemContext.sol
 - └─ ZKOSContractDeployer.sol
 - └─ ZKOSContractHelper.sol
 - └─ errors
 - └─ ZKOSContractErrors.sol
 - └─ interfaces
 - └─ IL2BaseTokenZKOS.sol
 - └─ IZKOSContractDeployer.sol
- └─ state-transition
 - └─ ChainTypeManagerBase.sol
 - └─ EraChainTypeManager.sol

```

├── IChainTypeManager.sol
├── L1StateTransitionErrors.sol
├── ZKsyncOSChainTypeManager.sol
├── chain-deps
│   ├── DiamondInit.sol
│   ├── StoredBatchHashing.sol
│   ├── ZKChainStorage.sol
│   └── facets
│       ├── Admin.sol
│       ├── Committer.sol
│       ├── Executor.sol
│       ├── Getters.sol
│       ├── IDiamondCut.sol
│       ├── Mailbox.sol
│       ├── Migrator.sol
│       └── ZKChainBase.sol
├── gateway-ctm-deployer
│   ├── GatewayCTMDeployer.sol
│   ├── GatewayCTMDeployerCTM.sol
│   ├── GatewayCTMDeployerCTMBase.sol
│   ├── GatewayCTMDeployerCTMZKsyncOS.sol
│   ├── GatewayCTMDeployerDA.sol
│   ├── GatewayCTMDeployerProxyAdmin.sol
│   ├── GatewayCTMDeployerValidatorTimelock.sol
│   ├── GatewayCTMDeployerVerifiers.sol
│   └── GatewayCTMDeployerVerifiersZKsyncOS.sol
├── chain-interfaces
├── data-availability
├── l2-deps
├── libraries
├── validators
├── transactionFilterer
│   └── GatewayTransactionFilterer.sol
├── l2-contracts/contracts
│   └── ForceDeployUpgrader.sol
├── system-contracts
│   ├── bootloader
│   │   └── bootloader.yul
│   └── contracts
│       ├── BootloaderUtilities.sol
│       ├── Compressor.sol
│       ├── Constants.sol
│       ├── ContractDeployer.sol
│       ├── Contracts.sol
│       ├── DefaultAccount.sol
│       ├── L1Messenger.sol
│       ├── SystemContext.sol
│       ├── SystemContractErrors.sol
│       ├── abstract/SystemContractBase.sol
│       ├── interfaces/*
│       └── libraries/*

```

Subsequently we performed a diff audit of the "matter-labs/era-contracts" repository with base commit [babbbed5](#) and head commit [dab8e51](#) and also reviewed [pull request #2182](#).

In scope were the following files:

```
contracts
├── l1-contracts/contracts/l2-upgrades
│   ├── L2ComplexUpgrader.sol
│   ├── L2GenesisForceDeploymentsHelper.sol
│   ├── L2GenesisUpgrade.sol
│   ├── L2V31Upgrade.sol
│   ├── SystemContractProxy.sol
│   ├── SystemContractProxyAdmin.sol
│   └── V31AcrossRecovery.sol
├── l1-contracts/contracts/upgrades
│   ├── BaseZkSyncUpgrade.sol
│   ├── BaseZkSyncUpgradeGenesis.sol
│   ├── BytecodesSupplier.sol
│   ├── EraSettlementLayerV31Upgrade.sol
│   ├── GovernanceUpgradeTimer.sol
│   ├── IL2V31Upgrade.sol
│   ├── L1FixedForceDeploymentsHelper.sol
│   ├── L1GenesisUpgrade.sol
│   ├── L1ZKsync0SV30Upgrade.sol
│   ├── L2UpgradeTxLib.sol
│   ├── UpgradeStageValidator.sol
│   ├── ZKsync0SSettlementLayerV31Upgrade.sol
│   └── ZkSyncUpgradeErrors.sol
```

We also reviewed [pull request #36](#) and [pull request #38](#) of the "zksync-association/zk-governance" repository.

System Overview

In this audit, we reviewed the changes introduced in ZKsync v3.1, which significantly expand interoperability capabilities, refine settlement-layer migration mechanics, and introduce structural changes to base token accounting and batch settlement. Notably, these updates enable permissionless execution of L1 -> L2 transactions, allowing ZKsync chains to achieve Stage 1 L2 status. While these improvements increase protocol flexibility and decentralization, they also introduce additional complexity in cross-chain execution and migration flows.

Interoperability Updates

The `InteropCenter` contract now holds funds collected as interop call fees, which can be claimed through a dedicated function by the coinbase address. This introduces a new balance-holding component within the interoperability layer.

Interop call recipients must implement a `receiveMessage` function and conform to the `IERC7786Recipient` interface. Currently:

- `L2AssetRouter` implements `receiveMessage` and receives interop calls for `finalizeDeposit`
- `InteropHandler` also implements `receiveMessage`, but only for bundle-related operations (unbundling, verification, execution)

Failed interop call refunds are not supported. Unlike failed L1 -> L2 transactions, which can be proven and refunded, failed interop calls result in funds remaining locked on the sending chain, where they are accounted for in the `pendingChainBalance` of the destination chain. This introduces new considerations around liveness and operational recovery.

The `InteropHandler` has also been extended with a new `unbundleBundle` function. Compared to `executeBundle`, this provides additional flexibility by allowing:

- Partial execution of bundle contents
- Multiple calls to process the same bundle incrementally
- Verification of bundles prior to the first unbundling call

Settlement and Migration Changes

Two new Diamond Proxy facets were introduced:

- `Committer`: Implements the precommit and commit stages of batch settlement
- `Migrator`: Provides preparation logic for settlement-layer migrations, including pausing deposits to synchronize priority trees between settlement layers

Regarding chain migrations:

- For the v3.1 upgrade, all chains initially settle on L1
- After the upgrade, chains may migrate L1 -> Gateway
- Chains may optionally migrate Gateway -> L1
- After returning to L1, no further migrations are allowed

Limiting migrations to two rounds reduces the complexity of chain and asset balance migration logic and simplifies reasoning about security guarantees.

Before migration, deposits on both settlement layers must be paused to synchronize their priority operation trees. A subtle detail is that during L1 → Gateway migration, deposits on L1 are not automatically unpaused after a successful migration; `L1Nullifier` must be explicitly called.

Base Token Architecture Changes

The base token architecture has been refactored into:

- `BaseTokenHolder`: Holds base token liquidity used for cross-chain transactions
- `L2BaseTokenBase`: Base token logic
- `L2BaseTokenEra`: ZKsyncEra-specific base token implementation

Cross-chain minting behavior now differs depending on transaction type:

- Interop transactions: `BaseTokenHolder::give` is called by `InteropHandler`
- L1 -> L2 transactions: `L2BaseTokenEra::mint` is called by the `bootloader`

Asset `chainBalance` Migration

Asset `chainBalance` migration is now performed lazily rather than automatically.

When a chain migrates to a new settlement layer:

- Asset balances are not immediately migrated
- The first interop or withdrawal transaction for an unmigrated asset will underflow and revert
- These reverting transactions signal that a chainBalance migration should be triggered manually

If an asset receives funds through interop or L1 deposits before migration, its chainBalance increases normally even if migration has not yet occurred.

Security Model and Trust Assumptions

The system and its configuration are complex, involving the setting and updating of critical parameters and, in certain cases, temporarily pausing parts of the protocol's functionality. While this upgrade enables ZKChains to activate the priority mode feature (allowing users to manually execute L1 -> L2 transactions) and thereby represents a significant step toward further decentralization, several privileged roles remain.

These roles, typically designated as owner, admin, or upgrader, and which are expected to differ across chains and contracts, retain the ability to modify configuration parameters, pause components, or otherwise affect core protocol behavior. They are also responsible for executing proper operational procedures during events such as upgrades, migrations, and emergency interventions.

As such, these roles were considered trusted entities within the scope of this audit. The security and correct functioning of the system therefore depend, to some extent, on the proper, timely, and honest operation of these privileged actors.

Below, we outline the actions expected to be executed correctly, which fall under these security assumptions:

- Across multiple contracts, the owner or admin roles have the ability to pause protocol functionality and are expected to do so under specific operational circumstances:
 - The owner of a chain's Diamond Proxy has the ability, and is expected, to pause [deposits](#) before a settlement layer migration begins. A delay window exists

between initiating and applying the pause. However, in the current implementation this delay is hardcoded to zero, resulting in the pause taking effect immediately.

This behavior does not impact chains planning to activate priority mode to achieve Stage 1 status, as pausing deposits for migration is not permitted for such chains.

However, a minor inconsistency may arise. An admin can first pause deposits and subsequently set the `canBeActivated` flag for priority mode. While this does not introduce a functional issue, since the flag is checked again when the migration actually begins, preventing migration for chains with priority mode enabled, it may still lead to a temporary state inconsistency from a view or monitoring perspective.

- The `InteropCenter` owner can [pause](#) the sending of interoperability messages and is expected to do so only when necessary. Even if paused indefinitely, users may be unable to bridge assets to other chains, but can still withdraw their funds to L1.
 - Owner entity of `ChainAssetHandlerBase` has the ability to pause/unpause migrations as well as the whole functionality of `ChainAssetHandler`.
 - Similarly, the owner of `L1Nullifier`, `AssetRouter` and `NativeTokenVault` can pause their functionalities
- The upgrader is responsible for setting critical system parameters (primarily addresses of dependent contracts) during upgrades and contract reinitialization.
 - Contract owners are also responsible for setting and updating important parameters, such as:
 - A chain's admin can set and update the [fee parameters](#).
 - The `BridgeHub` admin can [add or remove](#) chains from the list of whitelisted settlement layers. Therefore, this role is trusted to whitelist only legitimate chains that are not known to be compromised. Otherwise, chains that rely on this list could be put at risk by trusting a malicious settlement layer. At present, however, only L1 and the Gateway are whitelisted as settlement layers. This role is also responsible for [registering CTMs](#).
 - In the L1 -> GW chain migration case, L1 is not automatically informed that the chain migration has completed on GW. It is trusted that some entity will trigger `L1Nullifier.bridgeConfirmTransferResult`, for both failed and successful migrations, so that the L1 deposits can be unpaused.

Notes on the Test Suite

The codebase was accompanied by an extensive test suite. One minor observation concerns a testing pattern that caused some initial confusion. The protocol makes extensive use of cross-chain messages that are encoded on the source chain and decoded on the destination chain. To quickly verify that encoding and decoding are handled consistently, for example, that both sides use `abi.encode/abi.decode` or `abi.encodePacked/abi.decodePacked` rather than a mismatched pair, we looked to the tests as a reference. However, we observed that in some cases, such as the [test](#) for `GWAssetTracker::parseInteropCall`, the test manually constructs the expected encoded data inline rather than calling the actual encoding function `AssetRouterBase::getDepositCalldata` and ignoring the selector's bytes. This approach has two drawbacks. First, it is error-prone: a mismatch between the encoding method used in the test and the one used in production code, for instance, using `abi.encodePacked` on one side and `abi.decode` on the other, would go undetected. Second, by not invoking the canonical encoding function in the decode test, it becomes harder to trace encode/decode pairs across the codebase and confirm that they are symmetric.

Considerations on Compromised Chains and Malicious Settlement Layers

As part of the materials provided to support our understanding of the protocol and its security assumptions, the Matter Labs team shared research notes titled [“Malicious Settlement Layer \(SL\)”](#). These notes outline scenarios in which a ZKChain—whether acting as a settlement layer or not—may become compromised, and analyze whether such compromises can affect other chains in the ecosystem.

This section presents our interpretation of these assumptions and explains how we incorporated them into our audit methodology. In particular, we describe the analytical model we used to evaluate whether protocol features and cross-chain flows remain secure under the presence of compromised chains or malicious settlement layers.

While our analysis did not uncover fundamentally new attack vectors beyond those described in the provided notes, we identified a potential **cross-chain liveness failure scenario**. This scenario appears to contradict the claim that a compromised non-settlement-layer chain cannot affect the liveness of another chain, as it demonstrates conditions under which such an impact may occur via interoperability mechanisms.

At a high level, when a settlement layer is compromised, all chains settling on it are inherently at risk, and mitigation options are limited. However, certain pieces of information propagated across settlement layers—particularly during migration—can still be independently validated. In one of our reported findings (see *L-07 Missing Input Validation*), we highlight a case where additional verification could be introduced to improve robustness even under the assumption of a compromised settlement layer.

Isolation vs Propagation of Failures

The “Malicious Settlement Layer” notes analyze several failure scenarios, including compromised operators, broken proof systems, malicious Chain Type Managers (CTMs), or combinations thereof. These scenarios are evaluated based on whether their consequences remain confined to the compromised chain or propagate to other chains.

The primary impact discussed in the notes is “stealing funds.” However, in the presence of interoperability, this notion becomes more nuanced. To better reason about cross-chain effects, we refine this concept and introduce a more general framework for analyzing whether failures are **isolated** or **propagating**.

Consider two chains, **Chain A** and **Chain B**, both settling on the same settlement layer. Suppose Chain A is fully compromised, including both its operator and proof system.

In this setting, Chain A can submit an interop message corresponding to a transaction that never actually occurred. Since the proof system is compromised, this message is accepted as valid. When processed on Chain B, it results in the minting of bridged tokens without a corresponding burn on Chain A.

At first glance, this appears to create a cross-chain issue. However, the protocol’s accounting model—specifically the `GWAssetTracker`—ensures that global balances remain consistent. The malicious transaction decreases Chain A’s `chainBalance` and increases Chain B’s `chainBalance`. As a result, Chain A is effectively spending its own balance, and no value is created at the expense of other chains.

Thus, despite the malicious behavior, the consequences remain confined to Chain A.

To formalize this reasoning, we introduce the following abstraction:

We conceptually replace the compromised chain with an adversarial entity that can report arbitrary state transitions to the settlement layer. This entity does not violate protocol rules visible to other chains—it simply exercises its authority to declare state changes.

We then evaluate whether the observed outcome can be reproduced through *legitimate protocol interactions* under this model.

- If the same outcome can be achieved without violating protocol rules, the failure is considered **isolated**, as the compromised chain is only exercising authority it effectively already possesses.
- If the outcome cannot be reproduced without violating protocol invariants or impacting honest chains beyond expected behavior, the failure is considered **propagating**, indicating a broader protocol-level issue.

This framework provides a principled way to reason about cross-chain security, focusing not on individual attack scenarios but on whether a compromised chain can exceed its intended authority.

Cross-Chain Liveness Considerations

While the accounting model ensures isolation of value under compromise, liveness guarantees may be more subtle.

Consider a scenario where a **non-settlement-layer Chain A** is compromised (including both its operator and proof system). Under certain conditions, this compromised chain may affect the liveness of another chain, **Chain B**:

- Chain B's base token is native to Chain A
- Both chains share the same settlement layer (e.g., Gateway), enabling interoperability between them

Under these assumptions, it holds that initially:

- $\text{chainBalance[A][BASE]} = \text{uint256.max}$
- on B: $\text{balanceOf(BASE_TOKEN_HOLDER)} = \text{INITIAL_BASE_TOKEN_HOLDER_BALANCE}$
 $= (2^{127} - 1) < \text{uint256.max}$

The detailed attack scenario is described as follows:

1. Chain A submits a (fake) A->B interop TX transferring a huge BASE amount which would drain chain B's `BaseTokenHolder` balance.
2. Since the proof system is compromised, A's batch containing the interop TX settles successfully on GW, updating `sharedTree` to contain the new `chainRoot` of A, which contains the interop TX. This produces [a new interopRoot](#).
3. B's operator [includes](#) the new interopRoot in B's `L2InteropRootStorage`

4. Subsequently, on B, `InteropHandler` successfully [verifies](#) the proof of the interop bundle containing A's interop TX.
5. `InteropHandler` proceeds with executing the TX, [minting amount](#) and [sending it to the recipient's address](#).
6. After this transfer, the `BaseTokenHolder` contract has no BASE balance, so further BASE minting on B [will fail](#).

As a consequence, no more TXs can be processed on B, since the operator's fees [cannot be minted](#).

Medium Severity

M-01 Missing Pause Enforcement in `L2AssetRouter.bridgehubDepositBaseToken`

The `L2AssetRouter.bridgehubDepositBaseToken` function is not protected by the `whenNotPaused` modifier. Additionally, the internal function it invokes also lacks pause enforcement.

As a result, cross-chain transaction requests can still be initiated even when the asset router is in a paused state, bypassing the intended pause mechanism during emergency or maintenance scenarios.

Consider ensuring that `bridgehubDepositBaseToken` is protected by the `whenNotPaused` modifier to enforce consistent pause behavior across the contract.

Update: Resolved in [pull request #2141](#). The team stated:

For interop pausing on L2 is not ideal, since we don't have access to the L2. Pausing on GW is preferred.

Low Severity

L-01 Incorrect Length Validation in ERC-7930 Address Parsing

In `draft-InteroperableAddress.sol`, the `tryParseV1Calldata` function is responsible for validating calldata against the ERC-7930 format. However, toward the end of the function, [an inequality check is used](#) where a strict equality check would be more appropriate to enforce the exact expected structure. This may allow malformed inputs to pass validation.

Additionally, in `InteropCenter.sol`, both `_ensureEmptyChainReference` and `_ensureEmptyAddress` include a preliminary check that

`_interoperableAddress.length >= 5`. While [stricter validation is performed later](#), making this check functionally redundant, it appears intended as an early rejection mechanism. However, the minimum valid length for ERC-7930 addresses suggests that this condition should instead be `_interoperableAddress.length >= 6`.

Consider replacing the inequality check in `tryParseV1Calldata` with a strict equality check where appropriate to enforce correct calldata structure. Additionally, update the early length checks in `InteropCenter` to reflect the correct minimum length, or remove them if they are redundant with stricter downstream validation.

Update: Resolved in [pull request #2142](#).

L-02 Missing Early Validation of ERC-7930 Bundle Addresses

The `execution` and `unbundler` fields of the `BundleAttributes` struct are defined as bytes rather than addresses because, as indicated in the comments, they are expected to contain ERC-7930 addresses.

However, in `parseAttributes`, these fields are [simply decoded as bytes](#) without verifying that they conform to the expected ERC-7930 address format. The validation of this format is only performed later in `InteropHandler`.

As a result, it is possible to submit a bundle containing invalid execution and unbundler addresses. These malformed addresses will pass the initial parsing stage but will eventually cause the bundle execution to fail on the destination chain. Since users are not refunded when such transactions fail, this can lead to an unnecessary loss of funds and a degraded user experience.

Consider validating the execution and unbundler fields during attribute parsing, rather than deferring validation to `InteropHandler`. Rejecting malformed ERC-7930 addresses as early as possible will prevent invalid bundles from progressing and avoid unnecessary execution failures.

Update: Resolved in [pull request #2143](#).

L-03 Missing Validation of `interopCallValue` in Indirect Calls

Within `InteropCenter`, it is possible to specify a non-zero `interopCallValue` (representing the destination chain's base token amount) in an indirect call request. However, in practice, such requests are not supported by the NTV flow and will revert.

This behavior aligns with the intended design, where base token transfers should be performed via direct calls rather than indirect ones. Despite this, there is no explicit validation in `InteropCenter` enforcing that `interopCallValue == 0` for indirect calls.

Additionally, the `parseAttributes` logic does not prevent the simultaneous presence of both `interopCallValue` and `indirectCallValue`, allowing malformed or unsupported request configurations to pass initial validation.

A further edge case arises when considering custom `AssetHandler` implementations. A custom handler could potentially process both base and non-base token transfers within a single indirect call by consuming both `interopCallValue` and `indirectCallValue`. In such a scenario, `interopCallValue` may be accounted for twice: once within the custom handler logic (e.g., burn/lock) and again within `InteropCenter` via `totalBurnedCallsValue` and `_ensureCorrectTotalValue`, leading to inconsistent accounting.

No immediate impact is identified in the current implementation. However, the lack of explicit validation introduces ambiguity, increases the risk of misconfiguration, and may lead to incorrect accounting or double-counting in custom integrations.

Consider explicitly enforcing that `interopCallValue == 0` for indirect calls at the validation layer. Additionally, ensure that mutually incompatible attributes (such as `interopCallValue` and `indirectCallValue`) cannot coexist in the same request.

Update: Resolved in [pull request #2144](#). The team stated:

This issue is invalid as stated. InteropCenter intentionally supports generic indirect calls, including carrying source-chain msg.value into
initiateIndirectCall and specifying destination-side base-token value via interopCallValue. The limitation is only in the current standard bridge
implementation: L2AssetRouter/L2NativeTokenVault do not support non-zero destination msg.value in that indirect flow. So this is not a protocol-level

bug in `InteropCenter`, but at most an implementation-specific limitation of the current default `IL2CrossChainSender` path.

L-04 Potential Double Accounting in Base Token Bridging Flow

In the `L2NativeTokenVault` contract, the `_getTokenAndBridgeToChain` function (inherited from `NativeTokenVaultBase`) first calls `_handleBridgeToChain`. This function subsequently invokes `L2AssetTracker::handleInitiateBridgingOnL2`, which decreases `chainBalance` and updates `interopInfo.totalWithdrawalsToL1`, depending on the asset's origin and the L2 settlement layer.

Returning to `_getTokenAndBridgeToChain`, in the scenario where the token is the base token of the L2 and is bridged (i.e., not originating on the L2), execution reaches a branch where `BaseTokenHolder::burnAndStartBridging` is called. This function, in turn, calls `L2AssetTracker::handleInitiateBaseTokenBridgingOnL2`, which risks performing duplicate state updates to `interopInfo.totalWithdrawalsToL1`, since both `handleInitiateBridgingOnL2` and `handleInitiateBaseTokenBridgingOnL2` make the exact same call to `_handleInitiateBridgingOnL2Inner`.

Although this scenario does not currently occur in practice, since base token withdrawals do not follow the `NativeTokenVault` flow, the design remains error-prone and could introduce bugs in future logic updates or refactors.

To reduce this risk, consider avoiding the second call to `handleInitiateBaseTokenBridgingOnL2`, ensuring that accounting updates occur only once and preventing potential double-counting issues.

Update: Resolved in [pull request #2163](#).

L-05 Missing Timestamp Consistency Check in `Committer.verifyBatchTimestamp`

In the `verifyBatchTimestamp` function of the `Committer` facet, a `comment` states that all L2 blocks within a batch must have timestamps in the range `[batchTimestamp, l2LastBlockTimestamp]`.

In particular, this implicitly requires that `batchTimestamp ≤ l2LastBlockTimestamp`. However, this relationship is not explicitly validated in the current implementation. A batch

whose timestamps violate this constraint should never be accepted, as it represents a malformed batch. The absence of this check also weakens the intended freshness guarantees. Currently, the function verifies that:

- `l2LastBlockTimestamp` is not too large, and
- `batchTimestamp` is not too small

Given the expected ordering (`batchTimestamp ≤ l2LastBlockTimestamp`), these checks implicitly provide additional bounds in the opposite direction. Without explicitly enforcing this relationship, however, malformed batches with inconsistent timestamps may still pass validation.

Consider explicitly verifying that `batchTimestamp ≤ l2LastBlockTimestamp`. This simple check would strengthen the validation logic and prevent malformed batches from being accepted.

Update: Resolved in [pull request #2147](#).

L-06 Overly Broad Access Control Permissions Across Multiple Contracts

Several functions across the codebase implement access control checks that are broader than necessary, allowing multiple callers despite being invoked by only a subset of those authorized entities in practice.

The following instances were identified:

- In `Mailbox.sol`, the `bridgehubRequestL2Transaction` function [permits calls](#) from both `Bridgehub` and `InteropCenter`. In practice, this function is only invoked by `Bridgehub`, while `InteropCenter` exclusively interacts with `bridgehubRequestL2TransactionOnGateway`, making its access unnecessary.
- In `MessageRootBase.sol`, the `setMigratingChainBatchNumber` function uses the `onlyBridgehubOrChainAssetHandler` modifier, even though it is only ever called by the `ChainAssetHandler` contract. Allowing `Bridgehub` access expands permissions without functional justification.
- In `BaseTokenHolder.sol`, the `burnAndStartBridging` function is protected by the `onlyBridgingCaller` modifier, which permits access to `InteropHandler`

despite there being no observed execution path where `InteropHandler` invokes this function.

- In `L2BaseTokenEra.sol`, the `transferFromTo` function restricts access to a set of predefined addresses, including `L2BaseTokenHolder`. However, `BaseTokenHolder` does not directly invoke this function, as calls are routed through the `MsgValueSimulator` contract, making this permission appear unnecessary.

Although no immediate vulnerability arises from these misconfigurations, they violate the principle of least privilege and increase the attack surface in the event of future changes, integrations, or contract upgrades.

Consider removing unused authorized callers to tighten access control modifiers to include only the contracts that are strictly required to call each function.

Update: Resolved in [pull request #2148](#).

L-07 Missing Input Validation

Across the codebase, there are instances where function inputs are expected to satisfy certain properties, but these assumptions are not explicitly enforced. The following cases were identified:

- In `InteropCenter`, the `forwardTransactionOnGatewayWithBalanceChange` function overwrites the `balanceChange.baseTokenAssetId` parameter with the value stored within the Bridgehub contract, without checking that the provided value is the expected one.
- A chain that has been designated as a permanent rollup should not revert to a non-permanent state. In the Migrator facet, the `forwardedBridgeMint` function does not validate that, if `s.isPermanentRollup` is already set to true (i.e., the chain had previously been marked as a permanent rollup before migrating back to Gateway and now back to L1), the incoming `__commitment.isPermanentRollup` flag is also true. The function should revert if these values do not match.

A mismatch between these two values would only be possible in the case of a malicious or faulty settlement layer. However, since this is a straightforward validation, it is worth adding this extra check to strengthen invariants.

- In `Mailbox::requestL2TransactionToGatewayMailboxWithBalanceChange`, `_expirationTimestamp` is expected to always be 0, as noted in `IMailboxImpl`. However, this is not enforced.

Since the parameter is only used when computing the `canonicalTxHash`, allowing non-zero values introduces unnecessary flexibility and potential inconsistency. Enforcing `_expirationTimestamp == 0` would align with the intended design and improve determinism.

- In the `getL2UpgradeTxData` of the `EraSettlementLayerV31Upgrade` and `ZKSyncOSSettlementLayerV31Upgrade` contracts, consider checking that the `_zksyncOS` input agrees with the type of the contract (zkos or not). Also, consider checking that the selector of `_existingTxData` equals `IComplexUpgrader.forceDeployAndUpgrade`.

Consider adding the suggested input validations to strengthen assumptions, improve robustness, and enhance security.

Update: Resolved in [pull request #2149](#).

L-08 Asymmetric Base Token Supply Cap Between ZKOS and Non-ZKOS Chains

There is an asymmetry in the base token cap between ZKOS and non-ZKOS chains. In the ZKOS case, the total supply is computed as:

```
zkosPreV31TotalSupply + (INITIAL_BASE_TOKEN_HOLDER_BALANCE -  
L2_BASE_TOKEN_HOLDER_ADDR.balance)
```

while in the non-ZKOS case, it is computed as:

```
INITIAL_BASE_TOKEN_HOLDER_BALANCE -  
eraAccountBalance[L2_BASE_TOKEN_HOLDER_ADDR]
```

Both formulas yield the correct actual total supply since, in the non-ZKOS case, the already circulating total supply is deducted during initialization. However, the resulting caps differ between the two cases: `zkosPreV31TotalSupply +`

`INITIAL_BASE_TOKEN_HOLDER_BALANCE` for ZKOS, versus
`INITIAL_BASE_TOKEN_HOLDER_BALANCE` for non-ZKOS.

Consider making the caps symmetric in both cases to preserve the principle that all chains should behave consistently, regardless of whether they operate on top of ZKOS or not.

Update: Resolved in [pull request #2187](#).

Notes & Additional Information

N-01 Redundant Branching in `InteropCenter._ensureCorrectTotalValue`

The `_ensureCorrectTotalValue` function in `InteropCenter` handles `_totalBurnedCallsValue` differently depending on whether the amount corresponds to the chain's base token.

This distinction appears to be based on the assumption that the chain's base token must be handled [directly](#) through the base token contract, while non-base tokens must be handled through `AssetRouter.bridgehubDepositBaseToken`.

However, `bridgehubDepositBaseToken` [already supports](#) the case where the destination chain's base token is equal to the current chain's base token. As a result, the separate handling in `_ensureCorrectTotalValue` is unnecessary and introduces avoidable complexity.

Consider simplifying `_ensureCorrectTotalValue` by removing the unnecessary distinction and relying on the existing handling in `bridgehubDepositBaseToken` where appropriate. This would reduce complexity and improve readability without changing behavior.

Update: Acknowledged, will resolve.

N-02 Inconsistencies in Fee Price Adjustment Logic

In the `Admin` facet, the access-controlled `changeFeeParams` and `setTokenMultiplier` functions are used to update fee-related parameters on L1.

For chains that may activate Stage1 mode, such updates are expected to be constrained by `_enforcePriceChangeBound`. While `changeFeeParams` applies this bound conditionally only for chains that can activate Stage1 mode, `setTokenMultiplier` applies it unconditionally. This creates inconsistent behavior between the two fee update paths and unnecessarily restricts flexibility for chains that cannot activate Stage1 mode.

In addition, `_enforcePriceChangeBound` ensures that fee updates remain within predefined bounds relative to the previous price.

The upper bound is calculated as:

```
oldPrice * MAX_PRICE_CHANGE_NUMERATOR / MAX_PRICE_CHANGE_DENOMINATOR
```

With the current parameters (13/10), this is intended to allow up to a 30% increase. However, due to Solidity's integer division rounding down, the effective maximum increase is slightly below 30%.

For the lower bound, the ratio is inverted:

```
oldPrice * MAX_PRICE_CHANGE_DENOMINATOR / MAX_PRICE_CHANGE_NUMERATOR
```

This results in a factor of 10/13, corresponding to a decrease of approximately 23.08%, not 30%. Rounding does not compensate for this difference, leading to an inherent asymmetry between allowed increases and decreases. This may reduce flexibility when decreasing fees and introduce unintuitive behavior.

Consider aligning the bounds to achieve symmetric behavior if that is the intended design. Alternatively, explicitly document the asymmetry and its rationale to ensure correct expectations and usage. Furthermore, consider applying `_enforcePriceChangeBound` conditionally in `setTokenMultiplier` as well, to align its behavior with `changeFeeParams` and preserve flexibility where Stage1 mode is not applicable.

Update: Resolved in [pull request #2151](#).

N-03 InteropHandler Does Not Explicitly Inherit IERC7786Recipient

The `InteropHandler` contract is designed to receive ERC-7786 cross-chain messages and already implements the `receiveMessage` function required by `IERC7786Recipient`. However, it does not explicitly declare inheritance from the interface.

As a result, the contract is functionally compatible with the expected recipient flow, but it does not formally advertise ERC-7786 recipient compliance at the type level. This may reduce clarity and could affect integrations or tooling that rely on explicit interface inheritance rather than function-level compatibility alone.

Consider explicitly inheriting `IERC7786Recipient` in `InteropHandler` to make compliance with the expected recipient interface clear and intentional.

Update: Resolved in [pull request #2152](#).

N-04 Missing Revert on Invalid BUNDLE_IDENTIFIER in Interop Message Handling

In `GWAssetTracker`, the `_handleInteropCenterMessage` function [verifies](#) whether an incoming interop message contains the expected `BUNDLE_IDENTIFIER`. However, the function does not revert if the check fails.

The relevant comment in the code indicates that, starting from version 31, the failure of this validation is expected to cause a transaction revert. The current implementation does not enforce this behavior, resulting in a discrepancy between the code and the intended specification.

Consider updating the implementation to revert the transaction when the `BUNDLE_IDENTIFIER` check fails, ensuring alignment with the v.31 specification and preventing the processing of invalid messages.

Update: Resolved in [pull request #2153](#).

N-05 Ambiguous Accounting for Base Tokens in Asset Trackers

For L2 chains settling on L1, the `L1AssetTracker` and `L2AssetTracker` contracts maintain accounting of funds [deposited from](#) and [withdrawn to](#) L1. This accounting is used during migration to another settlement layer (e.g., GW), where token balances must be correctly [transferred](#).

The design assumes that the tracked L1 `chainBalance` may temporarily exceed the actual migratable amount due to unclaimed failed deposits. However, the implementation does not distinguish between base tokens and non-base tokens.

This distinction is relevant because failed L1 → L2 deposits behave differently for base tokens: instead of being claimed on L1, they are automatically [refunded](#) to the refundRecipient on L2. As a result, the accounting model implicitly treats all tokens uniformly, despite differences in their failure handling semantics.

No immediate impact is expected. However, the lack of differentiation introduces ambiguity in accounting assumptions, making the system more error-prone and harder to reason about, particularly during migrations.

Consider explicitly differentiating accounting logic between base and non-base tokens, or clearly document the assumptions that justify treating them uniformly. This will improve correctness guarantees and reduce the risk of accounting inconsistencies in future changes.

Update: Resolved in [pull request #2154](#).

N-06 Inconsistent Restoration of pausedDepositsTimestamp Across Migration Flows

Prior to initiating a chain migration, the

`Migrator.pauseDepositsBeforeInitiatingMigration` function is invoked to set the `s.pausedDepositsTimestamp` state variable within the DP storage [on both L1 and GW](#). This mechanism ensures deposits are paused consistently across domains during migration.

However, the restoration of this state variable post-migration is not handled symmetrically across all migration scenarios.

Specifically, the internal function `_unpauseDeposits`, which resets `s.pausedDepositsTimestamp`, is invoked in the following cases:

- Within `Migrator.forwardedBridgeMint`, upon completion of the migration process on the new SL.
- Within `Migrator.forwardedBridgeMintTransferResult`, intended to be executed on L1 following the completion of an L1 → GW migration (initiated via `L1Nullifier`).
- Within the access-controlled `unpauseDeposits` function, which is restricted to execution on GW.

In the case of a GW → L1 migration, none of the above execution paths result in `_unpauseDeposits` being called on GW. As a result, `s.pausedDepositsTimestamp` remains set on GW indefinitely after migration.

In the current system design, this inconsistency does not lead to immediate functional issues, as a GW → L1 migration is terminal and no further migrations are expected.

However, this asymmetric state restoration introduces state inconsistency across chains, which may complicate reasoning about system invariants and potentially lead to forward-compatibility risks, particularly if future versions support multiple migrations or bidirectional migration cycles involving GW.

Consider ensuring that `_unpauseDeposits` is invoked on GW following a GW → L1 migration to restore `s.pausedDepositsTimestamp` consistently across all migration paths.

Update: Acknowledged, will resolve.

N-07 Missing Deposit Pause Validation on Receiving Settlement Layer

Chain migration requires deposits to be paused on both the source and destination settlement layers to ensure consistency during the process.

In `Migrator.sol`, `forwardedBridgeBurn` validates that deposits are paused on the settlement layer the chain is migrating from. However, `forwardedBridgeMint`, which executes on the receiving settlement layer, does not perform a similar check.

While this does not introduce a direct issue in the current implementation, it creates an inconsistency in validation between the two migration steps and omits a useful sanity check.

Consider adding a deposit pause check in `forwardedBridgeMint` for consistency and improved robustness.

Update: Resolved in [pull request #2156](#).

N-08 Hardcoded `originChainId` in Base Token Finalization May Be Misleading

In `L2AssetTracker.sol`, the `handleFinalizeBaseTokenBridgingOnL2` function is invoked when a base token deposit is finalized. Since `L2AssetTracker` tracks `chainBalance` only for native tokens of the chain, and base tokens are in principle bridged assets, this function primarily handles forced asset migration and updates to `totalSuccessfulDepositsFromL1` when applicable.

Within this flow, the internal `_handleFinalizeBridgingOnL2Inner` function is called with a hardcoded `originChainId` set to `L1_CHAIN_ID`, regardless of the actual origin of the asset.

While this does not introduce a functional issue in the current design, the use of a hardcoded value that may not reflect the true origin can be misleading and may reduce code clarity.

Consider documenting explicitly that `L1_CHAIN_ID` is used as a placeholder in this context and does not necessarily reflect the actual origin of the asset. This will improve readability and reduce the risk of incorrect assumptions in future modifications.

Update: Resolved in [pull request #2157](#).

N-09 Naming Inconsistencies

A few naming choices in the codebase appear outdated or imprecise relative to the current architecture and upcoming functionality.

- In `Mailbox.sol`, `requestL2TransactionToGatewayMailboxWithBalanceChange` reverts with a `NotHyperchain_error` when the caller is not the diamond proxy of the relevant chain. The error name appears to follow older terminology and may no longer match the current ZKChain naming convention.

- In `AssetRouterBase`, the `_burn` function uses the variable name `l1AssetHandler` for the handler associated with `assetId`. This name implies that the handler is always an L1 contract, whereas starting from version 31, the same logic will also be used for interop transactions, wherein the resolved handler may be an L2 asset handler.

Consider renaming these identifiers to better reflect current terminology and behavior.

Update: Resolved in [pull request #2158](#).

N-10 Redundant Code and Minor Structural Improvements

Multiple instances of unused variables, redundant imports, dead code, and suboptimal code organization were identified across the codebase. While none of these introduce functional issues, they increase complexity and reduce maintainability.

In `IAssetTrackerBase.sol`:

- The `INTEROP_BALANCE_CHANGE_VERSION` seems to be unused throughout the codebase.

In `AssetTrackerBase.sol`:

- The `_sendL1ToGatewayMigrationDataToL1` internal function is only used within the `L2AssetTracker` contract and thus could be moved to that contract instead of having it in the base contract. Similarly, the `_sendGatewayToL1MigrationDataToL1` could be moved to the `GWAssetTracker` contract.

In `SystemContractBase.sol`:

- The `onlyCallFromBootloaderOrInteropHandler` and `onlyCallFromInteropCenterOrNTV` modifiers are not used throughout the codebase.

In `ChainAssetHandlerBase.sol`:

- In `_finalizeBridgeBurn`, `_recordMigrationToSL` is conditionally called only when `block.chainId` equals L1, but this check is redundant since `_recordMigrationToSL` is only implemented in `L1ChainAssetHandler`.

Consider removing unused constants, structs, or modifiers and eliminating redundant imports and dead code. In addition, consider relocating functions to the contracts where they are

actually used. These changes will improve readability, reduce complexity, and strengthen long-term maintainability.

Update: Resolved in [pull request #2159](#). The team stated:

We considered removing the `block.chainid == l1ChainId()` guard around `recordMigrationToSL`. While the L2 override is indeed a no-op, we decided to leave the explicit check for clearness of intention.

N-11 Typographical Errors

A few typos were identified in codebase comments. In particular:

- In `Config.sol`, `"support"` should be `"supported"`.
- In the comment of `IAssetHandler::bridgeMint`, `"malicioud"` should be `"malicious"`.
- In the comment describing the `L2ContractAddresses::L2_COMPLEX_UPGRADER_ADDR` constant, `"upgragedes"` should be `"upgrades"`.

Consider fixing these typos to improve readability and maintain code quality.

Update: Resolved in [pull request #2160](#).

N-12 Redundant Logic and Minor Code Simplifications

Several instances of redundant logic and unnecessary branching were identified across the codebase. While these do not introduce functional issues, they increase code complexity and reduce readability.

- In `GWAssetTracker._handleAssetRouterMessageInner`, the call to `_decreaseChainBalance(_sourceChainId, _assetId, amount)` is executed in both branches of the conditional logic. This operation can be moved outside the if-else blocks to eliminate duplication.
- In `AssetRouterBase._getTransferData`, the if-else statement checking the encoding version is unnecessary, as version validation is already handled within

`DataEncoding.decodeAssetRouterBridgehubDepositData`. The function can therefore be simplified into a single-line return.

- In `ChainAssetHandlerBase.bridgeMint`, the chain's new migration number `is validated` on L1 before calling `_recordMigrationFromSL`, even though the `same validation is already performed within _recordMigrationFromSL`, making the initial check redundant.

Consider refactoring the identified sections to remove redundant checks and unnecessary branching. Simplifying control flow will improve readability, reduce duplication, and lower the risk of inconsistencies in future changes.

Update: Resolved in [pull request #2161](#).

N-13 Misleading or Incomplete Comments

A few instances of misleading or incomplete comments were identified in the codebase:

- In `IMigrator.sol`, the `docstring` for `pauseDepositsBeforeInitiatingMigration` implies that the function is called only for L1->GW migrations, while in fact, it must be called also for GW->L1 migrations.
- Several `comments` in the `AssetRouter` contracts imply that they only support L1->L2 transactions, while with v.31 and onwards, they are also used for interop transactions between ZKChains.
- In `ChainAssetHandlerBase`, there is an `incomplete comment` inside `bridgeMint`.
- Within `ZKOSContractDeployer`, the `comment` describing the `onlyComplexUpgrader` modifier wrongly states that it ensures `NativeTokenVault` is the caller.
- The `docstrings` describing the parameters of `setBytecodeDetailsEVM` within `IZKOSContractDeployer` are not clear. `_bytecodeHash` and `_observableBytecodeHash` are described as "the hash of the bytecode" and "the hash of the observable bytecode," while the latter should more accurately state "the observable (keccak256) hash of the bytecode." Similar confusion exists in the description of the `_bytecodeLength` parameter.

- In the `fallback` of `L1MessengerZK0S`, a `comment` says that any ETH sent will be intentionally burnt, while the opcode `invalid()` is used that burns all the forwarded gas but reverts all other changes, including ETH sent with the transaction.
- The `comment` in `L2NativeTokenVaultZK0S::_deployBeaconProxy` falsely states that the `ContractDeployer` system contract is used.
- In `BaseZkSyncUpgradeGenesis::_setNewProtocolVersion`, in the comment about the `minorDelta` variable, the `>=` should be just `>`.

Consider updating these comments to improve clarity, accuracy, and overall readability.

Update: Resolved in [pull request #2162](#).

N-14 Missing Deadline Initialization Check in GovernanceUpgradeTimer

The `checkDeadline` function in `GovernanceUpgradeTimer` does not verify that a deadline has been set before evaluating it. If `checkDeadline` is called prior to a deadline being initialized, the function will silently operate on a zero or default value, potentially allowing the deadline check to pass unexpectedly.

Consider adding a sanity check at the beginning of `checkDeadline` that reverts if the deadline has not yet been set.

Update: Resolved in [pull request #2184](#).

N-15 Unused Code

Two instances of unused code were identified within the zksync-os module. Dead code increases the cognitive overhead of auditing and maintaining a codebase.

- The `ZK0SContractHelper` contract contains only an `IBaseToken` interface definition and does not appear to be imported or referenced anywhere in the codebase.
- The `L1MessengerSendFailed` custom error defined in `ZK0SContractErrors.sol` is never referenced within the codebase.
- In `BaseZkSyncUpgrade.sol`, the `UpgradeInnerFailed` error is unused.

Consider removing all instances of unused code for clarity and consistency.

Update: Resolved in [pull request #2185](#).

Client Reported

CR-01 Interop Bundles Can Be Claimed On Chains Settling On L1

A valid interop bundle can be executed or unbundled on a destination chain where interop claims should be disabled, because the gating check reads the wrong source of settlement-layer state.

Specifically, `InteropHandler` is intended to block interop claims while the destination chain is settling directly on L1. The check is implemented in `InteropHandler.executeBundle` and `InteropHandler.unbundleBundle`:

```
require(L2_BRIDGEHUB.settlementLayer(block.chainid) != L1_CHAIN_ID,  
CannotClaimInteropOnL1Settlement());
```

However, `L2_BRIDGEHUB.settlementLayer(block.chainid)` is not the current chain's runtime settlement layer. `settlementLayer` is Bridgehub's registry state — it tracks settlement-layer assignments for chains known to that Bridgehub, and is not guaranteed to contain a correct entry for `block.chainid` on the current L2. The runtime source of truth is `L2_SYSTEM_CONTEXT_SYSTEM_CONTRACT.currentSettlementLayerChainId()`, which is maintained by `SystemContext`.

On an L1-settled destination chain, `L2_BRIDGEHUB.settlementLayer(block.chainid)` may be unset (i.e. 0). In this case, the check will proceed even though the chain is settling directly on L1 — bypassing the invariant that interop claims may only happen while the chain settles on Gateway.

In `InteropHandler.executeBundle` and `InteropHandler.unbundleBundle` consider querying `SystemContext` about the chain's settlement layer instead of Bridgehub, to ensure that only chains which settle on Gateway are involved in interop operations.

Update: Resolved in [pull request #2188](#).

CR-02 ProtocolUpgradeHandler cannot freeze chains under the ZKOS ChainTypeManager

`ProtocolUpgradeHandler` holds a single `ChainTypeManager` (CTM) reference, set once at deployment as an immutable. This was sound while every ZK chain registered to a given Bridgehub shared a single CTM, but with the introduction of ZKsync OS this is no longer the case: Era chains and ZKsync OS chains live under different `ChainTypeManager` instances while remaining under the same `Bridgehub` and the same `ProtocolUpgradeHandler`. The `freeze` / `unfreeze` flows in `ProtocolUpgradeHandler` still assume one CTM and therefore cannot reach chains owned by the other.

In essence, any chain whose `ChainTypeManager` is not the one stored in `ProtocolUpgradeHandler`, the call to `freezeChain` / `unfreezeChain` will not succeed since the CTM has no record of that chainId. This failure is invisible in the freeze paths because the calls are wrapped in `try { ... } catch {}`, so the loops silently skip the chains they cannot reach and `_freeze` / `_unfreeze` still emit `Freeze` / `Unfreeze` events and complete successfully.

The consequence is a partial freeze: chains under the configured CTM are frozen, but chains under the other CTM remain operational. This breaks the security guarantee the freeze mechanism is meant to provide, since the Security Council can no longer halt the entire protocol via `ProtocolUpgradeHandler` during an emergency.

Update: Resolved in [pull request #42](#).

Conclusion

In this diff audit we reviewed a wide range of changes that have been introduced to the protocol, including updates to interoperability features, the introduction of permissionless execution of L1->L2 transactions to support Stage-1 L2 status for ZKChains, improvements to chain migration between settlement layers, and more refined balance accounting during migrations.

While the overall design is complex, the codebase was of very high quality, with thorough inline documentation and an extensive test suite covering the new functionalities. The issues identified during the audit were primarily related to edge cases in interoperability and migration flows, rather than fundamental design flaws.

The Matter Labs team was highly responsive, knowledgeable, and collaborative throughout the audit. They were readily available to answer questions and provide detailed explanations of design decisions, which significantly contributed to the efficiency and depth of the review.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.